

UNIVERSIDAD PERUANA UNIÓN
FACULTAD DE INGENIERÍA Y ARQUITECTURA
E.P. DE INGENIERÍA DE SISTEMAS



REPORTE APLICACIONES DISTRIBUIDAS

EXAMEN PARCIAL SEGURIDAD

Benjamin David Reyna Barreto

INTEGRANTES:

Orbezo Perkga Lee Brandon

Lima, Junio 2026


```

-- =====
-- TABLA DESTINO
-- =====
CREATE TABLE destino (
    iddestino BIGINT AUTO_INCREMENT PRIMARY KEY,
    ciudest VARCHAR(100) NOT NULL,
    costdest DECIMAL(10,2) NOT NULL
);

-- =====
-- TABLA ASIENTO
-- =====
CREATE TABLE asiento (
    idasiento BIGINT AUTO_INCREMENT PRIMARY KEY,
    idbus BIGINT NOT NULL,
    numasi VARCHAR(5) NOT NULL,
    estasi CHAR(1) NOT NULL,

    CONSTRAINT fk_asiento_bus
        FOREIGN KEY (idbus)
        REFERENCES bus(idbus)
);

-- =====
-- TABLA PROGRAMACION
-- =====
CREATE TABLE programacion (
    idprogramacion BIGINT AUTO_INCREMENT PRIMARY KEY,
    idbus BIGINT NOT NULL,
    fecha DATE NOT NULL,
    hora TIME NOT NULL,

    CONSTRAINT fk_programacion_bus
        FOREIGN KEY (idbus)
        REFERENCES bus(idbus)
);

-- =====
-- TABLA RESERVA
-- =====
CREATE TABLE reserva (
    idreserva BIGINT AUTO_INCREMENT PRIMARY KEY,
    fechareser DATE NOT NULL,
    horareser TIME NOT NULL,

    idcliente BIGINT NOT NULL,
    idprogramacion BIGINT NOT NULL,
    iddestino BIGINT NOT NULL,

    CONSTRAINT fk_reserva_cliente
        FOREIGN KEY (idcliente)
        REFERENCES cliente(idcliente),

    CONSTRAINT fk_reserva_programacion
        FOREIGN KEY (idprogramacion)
        REFERENCES programacion(idprogramacion),

    CONSTRAINT fk_reserva_destino
        FOREIGN KEY (iddestino)
        REFERENCES destino(iddestino)
);

```

• **DATOS DE PRUEBA**

```
INSERT INTO cliente(nomcli, apecli, edadcli, sexocli)
VALUES
('Juan','Perez',25,'M'),
('Maria','Lopez',30,'F');
```

```
INSERT INTO bus(modbus, placabus, capbus)
VALUES
('Mercedes Benz','ABC123',40),
('Volvo 9800','XYZ789',50);
```

```
INSERT INTO destino(ciudest, costdest)
VALUES
('Lima',35.00),
('Arequipa',80.00);
```

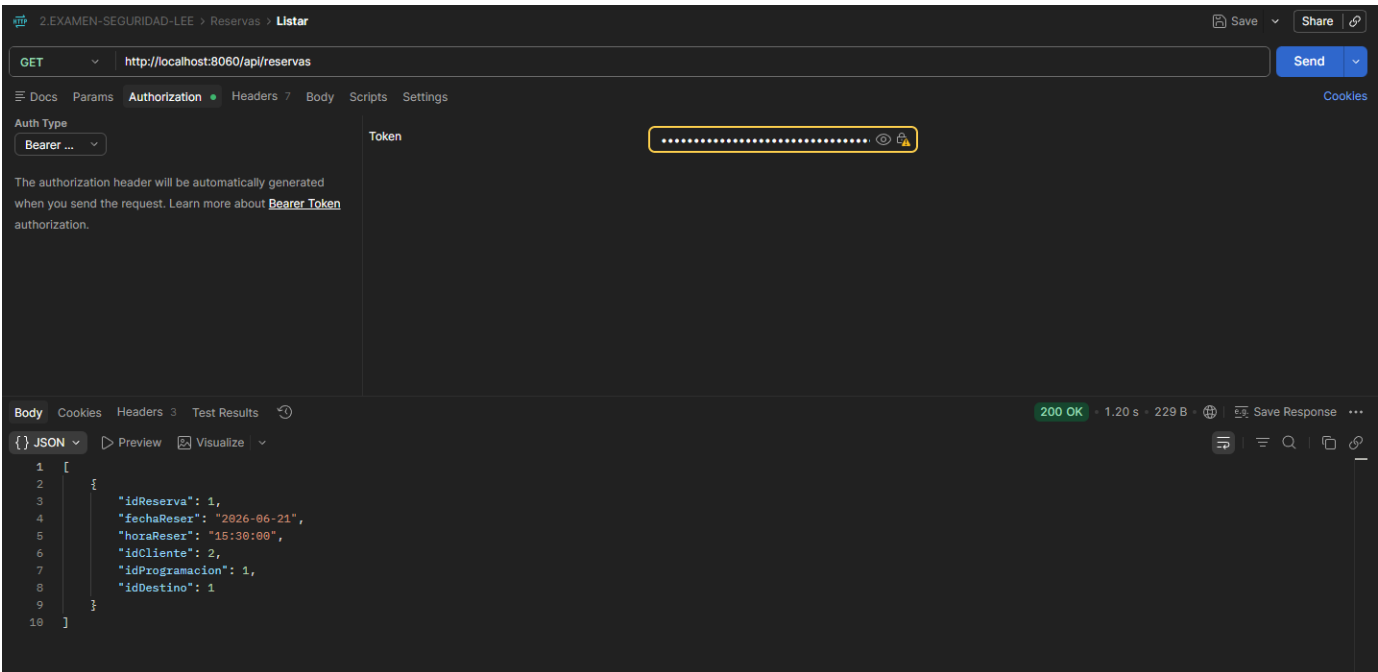
```
INSERT INTO asiento(idbus,numasi,estasi)
VALUES
(1,'A1','D'),
(1,'A2','D'),
(2,'B1','D');
```

```
INSERT INTO programacion(idbus,fecha,hora)
VALUES
(1,'2026-06-25','08:00:00'),
(2,'2026-06-26','10:30:00');
```

```
INSERT INTO reserva(fechareser,horareser,idcliente,idprogramacion,iddestino)
VALUES
('2026-06-21','15:00:00',1,1,1);
```

• **EVENTOS (CASO DE RESERVAS)**

Servicio Consumidor		Servicio Proveedor	Endpoint
ms-reserva		ms-cliente	GET /api/clientes/{id}
ms-reserva		ms-destino	GET /api/destinos/{id}
ms-reserva		ms-programacion	GET /api/programaciones/{id}



• PRUEBAS UNITARIAS

2.EXAMEN-SEGURIDAD-LEE > Cliente > Crear

POST http://localhost:8060/api/clientes

Body

```
1 {
2   "nomCli": "Giezy",
3   "apeCli": "Perez",
4   "edadCli": 25,
5   "sexoCli": "M"
6 }
```

201 Created · 702 ms · 197 B

Body

```
1 {
2   "idCliente": 3,
3   "nomCli": "Giezy",
4   "apeCli": "Perez",
5   "edadCli": 25,
6   "sexoCli": "M"
7 }
```

2.EXAMEN-SEGURIDAD-LEE > Bus > Listar

GET http://localhost:8060/api/buses

Auth Type: Bearer ...

Token: [Redacted]

200 OK · 679 ms · 185 B

Body

```
1 [
2   {
3     "idBus": 1,
4     "modBus": "Scania 2025",
5     "placaBus": "ABC-123",
6     "capBus": 40
7   }
8 ]
```

2.EXAMEN-SEGURIDAD-LEE > Destinos > Listar

GET http://localhost:8060/api/destinos

Auth Type: Bearer ...

Token: [Redacted]

200 OK · 723 ms · 171 B

Body

```
1 [
2   {
3     "idDestino": 1,
4     "ciuDest": "Arequipa",
5     "costDest": 45.50
6   }
7 ]
```

2. EXAMEN-SEGURIDAD-LEE > Asientos > **Listar**

GET http://localhost:8060/api/asientos Send

Docs Params **Authorization** Headers 7 Body Scripts Settings Cookies

Auth Type: Bearer ... Token:

The authorization header will be automatically generated when you send the request. Learn more about [Bearer Token](#) authorization.

Body Cookies Headers 3 Test Results 200 OK · 666 ms · 170 B Save Response

```
{
  "idAsiento": 1,
  "idBus": 1,
  "numAsi": "A1",
  "estAsi": "0"
}
```

2. EXAMEN-SEGURIDAD-LEE > Programaciones > **Listar**

GET http://localhost:8060/api/programaciones Send

Docs Params **Authorization** Headers 7 Body Scripts Settings Cookies

Auth Type: Bearer ... Token:

The authorization header will be automatically generated when you send the request. Learn more about [Bearer Token](#) authorization.

Body Cookies Headers 3 Test Results 200 OK · 695 ms · 187 B Save Response

```
{
  "idProgramacion": 1,
  "idBus": 1,
  "fecha": "2026-06-21",
  "hora": "15:30:00"
}
```

2. EXAMEN-SEGURIDAD-LEE > Reservas > **Listar**

GET http://localhost:8060/api/reservas Send

Docs Params **Authorization** Headers 7 Body Scripts Settings Cookies

Auth Type: Bearer ... Token:

The authorization header will be automatically generated when you send the request. Learn more about [Bearer Token](#) authorization.

Body Cookies Headers 3 Test Results 200 OK · 1.20 s · 229 B Save Response

```
[
  {
    "idReserva": 1,
    "fechaReser": "2026-06-21",
    "horaReser": "15:30:00",
    "idCliente": 2,
    "idProgramacion": 1,
    "idDestino": 1
  }
]
```

- **DESPLIEGUE DEL SISTEMA Y REGLAS DE NEGOCIO**

1. Despliegue del sistema

El sistema ha sido diseñado bajo una arquitectura de microservicios, desplegado mediante contenedores Docker y orquestado con Docker Compose.

La arquitectura incluye:

API Gateway como punto de entrada único

Service Registry (Eureka) para descubrimiento de servicios

Config Server para centralización de configuraciones

Microservicios independientes por dominio

Base de datos MySQL para persistencia

2. Flujo de comunicación del sistema

El flujo general del sistema es el siguiente:

Usuario → API Gateway → Microservicio correspondiente → Base de datos

Ejemplo:

Usuario → API Gateway → Auth Service → Usuario Service → MySQL

3. Reglas de negocio implementadas**3.1 Usuario**

Todo usuario debe autenticarse para acceder al sistema

Solo usuarios válidos pueden realizar operaciones dentro del sistema

3.2 Reservas

No se permite realizar reservas sin disponibilidad de asientos

Una reserva solo puede ser realizada por un usuario autenticado

Cada reserva debe estar asociada a un viaje específico

3.3 Asientos

Un asiento solo puede ser asignado a una única reserva

No se permite doble reserva de un mismo asiento

3.4 Programación de viajes

Un viaje debe tener una fecha válida de programación

No se permite programar viajes sin asignación de bus

3.5 Buses

Cada bus tiene una capacidad máxima de pasajeros

No se puede exceder la capacidad del bus

3.6 Seguridad

El sistema utiliza autenticación basada en JWT

Todas las peticiones protegidas deben incluir token válido

API Gateway valida el acceso a los microservicios